GLOBAL OPPORTUNITIES — COMPLETE FULL-STACK MASTER PROMPT

1. PROJECT OVERVIEW

Build a modern, premium, multilingual full-stack platform called **Global Opportunities**.

The platform helps individuals and organizations discover opportunities, understand funding calls, prepare projects, submit applications, receive funding, manage disbursements and payments, and monitor project implementation.

The platform must combine:

- opportunity discovery
- funding opportunities
- calls for projects
- project preparation and management
- applications
- evaluation and selection
- funding
- disbursement
- payment tracking
- document management
- notifications
- AI-assisted guidance
- multilingual access

The platform must be designed as a professional, trustworthy, scalable digital ecosystem for development opportunities.

---

2. CRITICAL ARCHITECTURAL RULE

There are two authoritative layers in this project.

FRONTEND / UI / UX AUTHORITY

The **original premium frontend prompt is authoritative for:**

- visual identity
- layout
- page composition
- navigation structure

- visual hierarchy
- interaction patterns
- animations
- cards
- sections
- dashboards
- responsive behavior
- accessibility behavior
- CTA placement
- premium design language
- user experience
- information architecture

Do **not** redesign or simplify the original frontend merely because the backend is an MVP.

The Global Opportunities content must be adapted to the original frontend structure while preserving its intended premium experience.

BACKEND / MVP AUTHORITY

The **latest Global Opportunities MVP is authoritative for:**

- backend modules
- entities
- database
- APIs
- business rules
- workflows
- permissions
- security
- payment logic
- funding logic
- application logic
- evaluation logic
- AI scope
- notifications
- documents
- infrastructure

Do not implement backend functionality that belongs only to future phases.

If the original UI contains a concept that is not part of the MVP, preserve its visual/layout position when appropriate, but represent it as:

- a future feature
- coming soon
- a disabled feature
- or a visual preview

Do not falsely expose unavailable backend functionality as operational.

---

3. TECHNOLOGY STACK — DO NOT CHANGE

The technologies already selected for the project are mandatory.

Frontend

Use:

- **Next.js**
- **React**
- **TypeScript**
- **Tailwind CSS**
- **React Hook Form**
- **Zod**
- **TanStack Query**

Do not replace these technologies with alternatives.

Backend

Use:

- **Django**
- **Django REST Framework**
- **PostgreSQL**
- **Redis**
- **Celery**
- **JWT Authentication**
- **DRF permissions**
- **DRF filtering**
- **DRF pagination**
- **OpenAPI / API documentation**

Do not replace Django/DRF/PostgreSQL/Redis/Celery with another backend stack.

Architecture

```
Next.js
   ↓
REST / JSON API
   ↓
Django REST Framework
```

```
   ↓
PostgreSQL
   ↓
Redis + Celery
   ↓
Object Storage
   ↓
AI Service
```

PostgreSQL is the primary source of truth.

---

4. PLATFORM VISION

Global Opportunities should make the complete opportunity-to-impact journey accessible through one platform:

Discover → Understand → Prepare → Apply → Evaluate → Decide → Fund → Pay → Monitor

The platform should reduce fragmentation between:

- applicants
- project owners
- organizations
- funders
- evaluators
- administrators

---

5. PRIMARY HERO MESSAGE

Use the main headline:

Discover Opportunities. Build Projects. Create Impact.

Supporting message:

Find funding opportunities and calls for projects, build your project, submit applications, access funding, and monitor implementation — all in one multilingual platform.

Primary CTA:

Explore Opportunities

Secondary CTA:

Get Started

---

6. MULTILINGUAL EXPERIENCE

The platform must support:

- English
- Français
- Kirundi
- Kiswahili

Language selection must be available from the main navigation.

The interface, validation messages, notifications, help content, and AI Assistant should support these languages where applicable.

Use localized message structures rather than hard-coded UI strings.

Frontend message structure:

```
messages/
├── en/
├── fr/
├── rn/
└── sw/
```

---

7. ORIGINAL PREMIUM NAVIGATION

Preserve the original premium navigation structure and behavior.

Main navigation:

- Home
- Opportunities
- Funding
- Projects
- Jobs
- Training
- About

- AI Assistant
- Language selector
- Login
- Get Started

However, only MVP-backed functionality must be operational.

Features outside the MVP, such as Jobs and advanced Training, may remain visible as:

- Coming Soon
- Future Feature
- preview cards

without requiring an active backend implementation.

The navigation must remain visually consistent with the original premium design.

---

8. PREMIUM LANDING PAGE

Preserve the original premium landing-page structure and behavior.

The landing page should feel:

- modern
- premium
- trustworthy
- professional
- accessible
- global
- development-oriented
- technology-driven

Use:

- strong typography
- generous spacing
- modern cards
- subtle gradients
- polished transitions
- hover effects
- responsive layouts
- clear CTA hierarchy
- professional illustrations/icons
- meaningful micro-interactions

Avoid:

- generic templates
- excessive visual clutter
- unnecessary decoration
- inconsistent components
- misleading functionality

---

9. LANDING PAGE — STICKY NAVBAR

Create a sticky responsive navigation bar.

Behavior:

- remains visible while scrolling
- adapts to mobile
- supports language selection
- clearly separates authentication actions
- provides smooth navigation
- supports dropdown/mobile navigation where appropriate

The navbar must follow the original premium layout.

---

10. LANDING PAGE — HERO

Create a large premium hero section.

Content:

Headline

Discover Opportunities. Build Projects. Create Impact.

Description

Find funding opportunities and calls for projects, build your project, submit applications, access funding, and monitor implementation — all in one multilingual platform.

CTAs

Explore Opportunities

Get Started

The hero may include:

- opportunity search preview
- platform dashboard preview
- visual project/funding illustration
- animated background elements
- premium motion

The hero must immediately communicate the platform's purpose.

---

11. TRUST / SOCIAL PROOF SECTION

Preserve the original trust/social-proof area.

Use appropriate platform-oriented messaging such as:

- Trusted opportunity ecosystem
- Secure application process
- Transparent evaluation
- Multilingual access
- Structured funding management

Do not invent real partners, donors, statistics, or testimonials.

If no verified partners exist, use neutral platform statements rather than fabricated logos.

---

12. OPPORTUNITY DISCOVERY

Create a premium opportunity-discovery section.

Users should be able to visually discover:

- Funding Opportunities
- Calls for Projects

Provide:

- search
- filters
- category/sector
- location
- funding range
- deadline

- status
- opportunity type

Cards should display:

- title
- provider
- funding amount
- currency
- location
- deadline
- status
- opportunity type
- CTA

---

13. INTELLIGENT OPPORTUNITY SEARCH

Preserve the original intelligent-search experience.

The interface should make searching feel simple and modern.

Examples:

- search by keyword
- filter by sector
- filter by location
- filter by funding amount
- filter by deadline
- filter by organization type

The search interface should be responsive and accessible.

---

14. FUNDING OPPORTUNITIES

Provide dedicated funding opportunity experiences.

Users can:

- browse funding opportunities
- search
- filter
- view details
- review eligibility

- review required documents
- review deadlines
- review funding amounts

Funding types:

- GRANT
- DONATION

---

15. CALLS FOR PROJECTS

Provide dedicated calls-for-projects experiences.

Users can view:

- title
- reference number
- objectives
- budget
- minimum funding
- maximum funding
- application deadline
- evaluation dates
- result date
- eligibility criteria
- evaluation criteria
- application process

---

16. AI ASSISTANT PREVIEW

Preserve the original AI Assistant section.

The AI Assistant must help users:

- understand opportunities
- understand eligibility
- identify required documents
- explain application procedures
- translate information
- simplify complex content
- guide users through the application process

Example query:

“Noshobora gusaba aya mafaranga?”

The AI should use official platform information.

It must never:

- invent eligibility rules
- invent deadlines
- invent funding amounts
- fabricate documents
- make final eligibility decisions
- make final selection decisions
- decide whether funding should be granted

---

17. HOW IT WORKS

Preserve the original premium "How It Works" section.

Display the platform workflow:

1. Discover

Find relevant opportunities and calls.

2. Understand

Review requirements, eligibility, documents, deadlines and funding information.

3. Prepare

Create and prepare a project.

4. Apply

Submit an application and required documents.

5. Evaluate

Applications are reviewed according to defined criteria.

6. Decide

Applications receive a selection decision.

7. Fund

Approved projects receive funding.

8. Pay

Disbursements and payments are tracked.

9. Monitor

Project implementation and activities can be followed.

---

18. PROJECT MANAGEMENT EXPERIENCE

Preserve the original premium project-management presentation.

Users should be able to create and manage projects containing:

- problem statement
- objectives
- expected results
- sector
- project type
- target population
- location
- dates
- budget
- requested funding
- own contribution
- activities
- progress
- status

Project activities should visually communicate progress.

---

19. PROJECT DASHBOARD PREVIEW

Create a premium dashboard preview showing:

- project status
- progress
- budget
- requested funding

- activities
- application status
- funding status
- upcoming deadlines

The visual design should follow the original dashboard behavior and hierarchy.

---

20. FUNDING ECOSYSTEM

Preserve the original funding ecosystem presentation while limiting operational functionality to the MVP.

The interface should communicate the relationship between:

Opportunity → Project → Application → Decision → Funding → Disbursement → Payment

Funding information may include:

- approved amount
- conditions
- funding period
- disbursement schedule
- payment status

---

21. ORGANIZATIONS

Organizations are central actors.

Organizations can have:

- profile
- registration information
- description
- contact details
- members
- verification status

Organizations may participate as:

- opportunity providers
- funders
- project owners
- applicants

according to their permissions and roles.

---

22. INDIVIDUALS / PROJECT OWNERS

Individuals can:

- create profiles
- discover opportunities
- create projects
- submit applications
- upload documents
- track applications
- receive notifications
- follow funding/payment information

---

23. PROJECT SHOWCASE

Preserve the visual/project-showcase pattern from the original premium prompt.

For the MVP, this should primarily communicate project information and project management rather than become a separate public social network.

Do not implement advanced social/community functionality unless included in the MVP.

---

24. BENEFITS SECTION

Present platform benefits such as:

For applicants

- discover opportunities
- understand requirements
- prepare projects
- submit applications
- track decisions

For organizations

- publish opportunities
- manage calls
- manage applications

- evaluate projects
- manage funding

For evaluators

- receive assignments
- review applications
- score criteria
- submit comments

For administrators

- manage users
- manage roles
- verify documents
- supervise platform activity
- audit sensitive actions

---

25. MULTILINGUAL SECTION

Create a dedicated premium multilingual section.

Explain that users can access the platform in:

- English
- French
- Kirundi
- Kiswahili

The language selector must be available globally.

The AI Assistant must understand and respond in the supported languages.

---

26. TESTIMONIAL SECTION

Preserve the original visual testimonial section if required by the original UI.

Do not invent real users or fabricated testimonials.

Use clearly marked placeholder content during development if authentic testimonials are unavailable.

---

27. IMPACT SECTION

Preserve the original premium impact section.

Do not invent numerical impact statistics.

Use qualitative messaging or clearly marked placeholders until verified data exists.

Examples:

- Opportunities made easier to discover
- Projects better prepared
- Applications more structured
- Funding processes more transparent
- Multilingual access expanded

---

28. PRICING SECTION

Preserve the original premium pricing-section layout if it is part of the original UI.

However, pricing must not imply a finalized commercial model unless one has been defined.

Use:

- informational presentation
- placeholder plans
- "Coming Soon"

where appropriate.

Do not create payment/subscription backend functionality as part of the MVP unless explicitly required later.

---

29. FAQ

Include FAQs covering:

- What is Global Opportunities?
- Who can use it?
- How can I find an opportunity?
- How do I create a project?
- How do I apply?
- What documents may be required?

- How does evaluation work?
- How is funding tracked?
- How are payments tracked?
- Which languages are supported?
- How does the AI Assistant work?

---

30. FINAL CTA

Create a strong final CTA.

Example:

Ready to turn opportunities into impact?

Buttons:

Explore Opportunities

Get Started

Preserve the original premium CTA layout and behavior.

---

31. FOOTER

Create a complete professional footer.

Include:

- platform description
- navigation
- opportunities
- projects
- resources
- About
- AI Assistant
- language selector
- legal links
- privacy
- terms
- contact information

Do not invent unsupported contact details.

32. AUTHENTICATION

Implement:

- registration
- login
- logout
- email verification
- password reset
- JWT authentication
- profile management

Use a custom Django user model.

---

33. USER ROLES

MVP roles:

- ADMIN
- APPLICANT
- ORGANIZATION_USER
- EVALUATOR

Visitors remain unauthenticated users.

Implement RBAC using Django/DRF permissions.

---

34. PERSON

```
PERSON
- id
- first_name
- last_name
- middle_name
- date_of_birth
- gender
- phone
- email
- nationality
- address
- province
```

```
- commune
- colline
- profile_photo
- created_at
- updated_at
```

---

35. USER

```
USER
- id
- person_id
- email
- password_hash
- status
- email_verified_at
- last_login_at
- created_at
- updated_at
```

Statuses:

- PENDING
- ACTIVE
- SUSPENDED
- BLOCKED
- DEACTIVATED

Constraints:

- email UNIQUE
- person_id UNIQUE

---

36. ROLE

```
ROLE
- id
- code
- name
- description
```

---

37. USER_ROLE

```
USER_ROLE
- id
- user_id
- role_id
- assigned_at
- assigned_by
```

Constraint:

```
UNIQUE(user_id, role_id)
```

---

38. ORGANIZATION

```
ORGANIZATION
- id
- name
- organization_type
- registration_number
- description
- email
- phone
- website
- address
- province
- commune
- verification_status
- created_at
- updated_at
```

Organization types:

- GOVERNMENT
- PUBLIC_INSTITUTION
- PRIVATE_COMPANY
- NGO
- NON_PROFIT
- ASSOCIATION
- COOPERATIVE
- FOUNDATION

- FINANCIAL_INSTITUTION
- EDUCATIONAL_INSTITUTION
- RESEARCH_INSTITUTION
- OTHER

Verification statuses:

- PENDING
- UNDER_REVIEW
- VERIFIED
- REJECTED
- SUSPENDED

---

39. ORGANIZATION_MEMBER

```
ORGANIZATION_MEMBER
- id
- organization_id
- person_id
- position
- is_primary_contact
- joined_at
- left_at
- created_at
- updated_at
```

Constraint:

```
UNIQUE(organization_id, person_id)
```

---

40. OPPORTUNITY MODEL

Use a generic opportunity concept to unify the frontend discovery experience.

The backend may use inheritance/OneToOne specialization so that:

```
OPPORTUNITY
   ├── FUNDING_OPPORTUNITY
   └── CALL_FOR_PROJECTS
```

Avoid maintaining duplicated independent opportunity records.

The generic opportunity contains:

```
OPPORTUNITY
- id
- provider_organization_id
- title
- description
- opportunity_type
- location
- online
- application_start_date
- application_deadline
- status
- created_by_user_id
- created_at
- updated_at
```

Types:

- FUNDING
- CALL_FOR_PROJECTS

Statuses:

- DRAFT
- PUBLISHED
- OPEN
- CLOSED
- CANCELLED
- ARCHIVED

---

41. FUNDING OPPORTUNITY

```
FUNDING_OPPORTUNITY
- id
- organization_id
- reference_number
- title
- description
- funding_type
- minimum_amount
```

```
- maximum_amount
- currency
- target_sectors
- target_population
- location
- application_start_date
- application_deadline
- status
- created_by_user_id
- created_at
- updated_at
```

Funding types:

- GRANT
- DONATION

---

42. FUNDING ELIGIBILITY CRITERION

```
FUNDING_ELIGIBILITY_CRITERION
- id
- funding_opportunity_id
- criterion_type
- name
- description
- mandatory
- created_at
- updated_at
```

Types:

- SECTOR
- LOCATION
- ORGANIZATION_TYPE
- PROJECT_TYPE
- EXPERIENCE
- BUDGET
- TARGET_POPULATION
- LEGAL_STATUS
- DOCUMENT
- OTHER

---

43. CALL FOR PROJECTS

```
CALL_FOR_PROJECTS
- id
- organization_id
- reference_number
- title
- description
- objectives
- budget_total
- minimum_funding
- maximum_funding
- currency
- launch_date
- application_deadline
- evaluation_start_date
- result_date
- status
- created_by_user_id
- created_at
- updated_at
```

Statuses:

- DRAFT
- PUBLISHED
- OPEN
- CLOSED
- UNDER_EVALUATION
- RESULTS_PUBLISHED
- CANCELLED

---

44. ELIGIBILITY CRITERION

```
ELIGIBILITY_CRITERION
- id
- call_id
- name
- description
- mandatory
- created_at
- updated_at
```

45. EVALUATION CRITERION

```
EVALUATION_CRITERION
- id
- call_id
- name
- description
- weight
- maximum_score
- created_at
- updated_at
```

46. PROJECT

```
PROJECT
- id
- reference_number
- title
- description
- problem_statement
- objectives
- expected_results
- project_type
- sector
- target_population
- location
- start_date
- end_date
- total_budget
- requested_amount
- own_contribution
- currency
- status
- created_by_user_id
- created_at
- updated_at
```

Project types:

- SOCIAL
- ECONOMIC

- AGRICULTURAL
- EDUCATIONAL
- HEALTH
- ENVIRONMENTAL
- TECHNOLOGY
- INFRASTRUCTURE
- ENTREPRENEURSHIP
- COMMUNITY_DEVELOPMENT
- RESEARCH
- INNOVATION
- OTHER

Statuses:

- DRAFT
- SUBMITTED
- UNDER_EVALUATION
- SHORTLISTED
- SELECTED
- REJECTED
- FUNDED
- IN_PROGRESS
- SUSPENDED
- COMPLETED
- CLOSED
- CANCELLED

---

47. PROJECT OWNER

```
PROJECT_OWNER
- id
- project_id
- person_id
- organization_id
- role
- is_primary
- start_date
- end_date
```

Exactly one of:

```
person_id
organization_id
```

must be populated.

---

48. PROJECT ACTIVITY

```
PROJECT_ACTIVITY
- id
- project_id
- name
- description
- planned_start
- planned_end
- actual_start
- actual_end
- progress_percentage
- status
- created_at
- updated_at
```

Statuses:

- PLANNED
- IN_PROGRESS
- COMPLETED
- DELAYED
- CANCELLED

---

49. APPLICATION

```
APPLICATION
- id
- call_id
- project_id
- submitted_by_user_id
- submitted_at
- status
- final_score
- decision
```

```
- decision_date
- decision_comment
- created_at
- updated_at
```

Statuses:

- DRAFT
- SUBMITTED
- ELIGIBILITY_REVIEW
- ELIGIBLE
- INELIGIBLE
- UNDER_EVALUATION
- SHORTLISTED
- SELECTED
- REJECTED
- WITHDRAWN

Constraint:

```
UNIQUE(call_id, project_id)
```

---

50. EVALUATION

```
EVALUATION
- id
- application_id
- evaluator_user_id
- started_at
- completed_at
- total_score
- general_comment
- status
- created_at
- updated_at
```

Statuses:

- ASSIGNED
- IN_PROGRESS
- COMPLETED
- CANCELLED

51. EVALUATION SCORE

```
EVALUATION_SCORE
- id
- evaluation_id
- criterion_id
- score
- comment
- created_at
- updated_at
```

Constraint:

```
UNIQUE(evaluation_id, criterion_id)
```

Score must satisfy:

```
0 <= score <= maximum_score
```

---

52. SELECTION DECISION

```
SELECTION_DECISION
- id
- application_id
- decision
- decision_date
- decision_reason
- decided_by_user_id
- created_at
- updated_at
```

Decisions:

- SELECTED
- REJECTED
- WAITLISTED

---

53. FUNDING

```
FUNDING
- id
- project_id
- provider_organization_id
- funding_type
- amount
- approved_amount
- currency
- approval_date
- start_date
- end_date
- status
- conditions
- created_by_user_id
- created_at
- updated_at
```

Types:

- GRANT
- DONATION

Statuses:

- PENDING
- APPROVED
- ACTIVE
- SUSPENDED
- COMPLETED
- CANCELLED
- CLOSED

Rule:

```
approved_amount <= amount
```

---

54. DISBURSEMENT

```
DISBURSEMENT
- id
- funding_id
- amount
- scheduled_date
- actual_date
- status
- created_at
- updated_at
```

Statuses:

- SCHEDULED
- PROCESSING
- COMPLETED
- FAILED
- CANCELLED

Constraint:

```
sum(disbursements) <= approved_amount
```

---

55. PAYMENT

```
PAYMENT
- id
- disbursement_id
- payer_organization_id
- payee_person_id
- payee_organization_id
- amount
- currency
- payment_method
- transaction_reference
- payment_date
- status
- created_by_user_id
```

```
- created_at
- updated_at
```

Methods:

- BANK_TRANSFER
- MOBILE_MONEY
- CASH
- CARD
- CHEQUE
- OTHER

Statuses:

- PENDING
- PROCESSING
- COMPLETED
- FAILED
- CANCELLED
- REFUNDED

Exactly one of:

```
payee_person_id
payee_organization_id
```

must be populated.

Constraint:

```
sum(payments) <= disbursement.amount
```

Use:

```
DecimalField(max_digits=18, decimal_places=2)
```

Never use floating-point values for financial amounts.

---

56. DOCUMENT

```
DOCUMENT
- id
- owner_person_id
- owner_organization_id
- document_type
- file_name
- file_url
- mime_type
- file_size
- verification_status
- verified_by_user_id
- verified_at
- uploaded_by_user_id
- uploaded_at
- expires_at
- created_at
- updated_at
```

Document types:

- IDENTITY
- ORGANIZATION_REGISTRATION
- PROJECT_PROPOSAL
- BUSINESS_PLAN
- BUDGET
- FINANCIAL_DOCUMENT
- CERTIFICATE
- CV
- SUPPORTING_DOCUMENT
- OTHER

Verification:

- PENDING
- UNDER_REVIEW
- VERIFIED
- REJECTED
- EXPIRED

---

57. DOCUMENT ATTACHMENT

```
DOCUMENT_ATTACHMENT
- id
- document_id
- entity_type
- entity_id
- attachment_type
- created_at
```

Supported targets include:

- PROJECT
- APPLICATION
- CALL_FOR_PROJECTS
- FUNDING_OPPORTUNITY
- ORGANIZATION
- FUNDING

---

58. NOTIFICATION

```
NOTIFICATION
- id
- recipient_user_id
- type
- title
- message
- is_read
- created_at
- read_at
```

Types:

- SYSTEM
- APPLICATION
- EVALUATION
- FUNDING
- DISBURSEMENT
- PAYMENT
- DOCUMENT
- DEADLINE
- PROJECT

- OTHER

---

59. AUDIT LOG

```
AUDIT_LOG
- id
- user_id
- action
- entity_type
- entity_id
- old_values
- new_values
- ip_address
- user_agent
- created_at
```

Audit logs are append-only.

Examples:

- CREATE_FUNDING
- APPROVE_FUNDING
- CREATE_DISBURSEMENT
- PROCESS_PAYMENT
- COMPLETE_PAYMENT
- SELECT_APPLICATION
- REJECT_APPLICATION
- UPDATE_USER_ROLE
- VERIFY_DOCUMENT

---

60. CORE RELATIONSHIPS

```
PERSON 1 ——— 0..1 USER

USER N ——— N ROLE
through USER_ROLE

PERSON N ——— N ORGANIZATION
through ORGANIZATION_MEMBER

ORGANIZATION 1 ——— N FUNDING_OPPORTUNITY
```

```
FUNDING_OPPORTUNITY 1 ——— N FUNDING_ELIGIBILITY_CRITERION

ORGANIZATION 1 ——— N CALL_FOR_PROJECTS

CALL_FOR_PROJECTS 1 ——— N ELIGIBILITY_CRITERION

CALL_FOR_PROJECTS 1 ——— N EVALUATION_CRITERION

CALL_FOR_PROJECTS 1 ——— N APPLICATION

PROJECT 1 ——— N PROJECT_OWNER

PROJECT 1 ——— N PROJECT_ACTIVITY

PROJECT 1 ——— N APPLICATION

APPLICATION 1 ——— N EVALUATION

EVALUATION 1 ——— N EVALUATION_SCORE

APPLICATION 1 ——— N SELECTION_DECISION

PROJECT 1 ——— N FUNDING

FUNDING 1 ——— N DISBURSEMENT

DISBURSEMENT 1 ——— N PAYMENT

USER 1 ——— N NOTIFICATION

USER 1 ——— N AUDIT_LOG

DOCUMENT 1 ——— N DOCUMENT_ATTACHMENT
```

---

61. DATA INTEGRITY

Implement:

- UNIQUE constraints
- foreign keys
- CHECK constraints
- validation
- transaction management

- authorization checks

Financial rules must always be enforced server-side.

---

62. DATE VALIDATION

Ensure:

```
application_start_date <= application_deadline
project.start_date <= project.end_date
funding.start_date <= funding.end_date
```

Disbursement scheduling must respect funding lifecycle rules.

---

63. FINANCIAL TRANSACTIONS

All financial operations must be:

- transactional
- validated
- authorized
- auditable
- protected from double processing

Use database transactions where appropriate.

---

64. DOCUMENT SECURITY

Documents must:

- use controlled access
- validate file types
- validate file size
- use secure storage
- record uploader
- support verification
- support expiration
- prevent unauthorized access

---

65. NOTIFICATIONS

Notifications should be triggered for important events such as:

- application submitted
- application status changed
- evaluator assigned
- evaluation completed
- selection decision
- funding approved
- disbursement scheduled
- payment completed
- document verification
- upcoming deadlines

Use Celery for asynchronous tasks.

---

66. AI ARCHITECTURE

Use:

```
USER
 ↓
AI API
 ↓
Authorization
 ↓
Platform Services
 ↓
Official Database
 ↓
Relevant Opportunity / Call / Project Data
 ↓
Eligibility Service
 ↓
AI Explanation
 ↓
USER
```

The AI must not become a second source of truth.

---

67. AI SAFETY AND ACCURACY

The AI Assistant must:

- use official platform data
- explain information
- cite or identify the relevant platform information when appropriate
- simplify complex requirements
- translate content
- identify missing documents
- explain application procedures

It must never fabricate:

- funding amounts
- deadlines
- eligibility criteria
- documents
- decisions

It must never make final:

- eligibility decisions
- selection decisions
- funding decisions

---

68. FRONTEND ROUTES

Public

```
/
/about
/opportunities
/opportunities/[id]
/funding-opportunities
/funding-opportunities/[id]
/calls
/calls/[id]
```

Authentication

```
/auth/login
/auth/register
/auth/verify-email
/auth/forgot-password
/auth/reset-password
```

Authenticated

```
/dashboard
/profile
/organizations
/organizations/create
/organizations/[id]
/projects
/projects/create
/projects/[id]
/applications
/applications/[id]
/evaluations
/evaluations/[id]
/funding
/funding/[id]
/payments
/payments/[id]
/documents
/notifications
/ai
```

Future routes may be introduced later for:

```
/jobs
/training
```

but they must not be presented as implemented MVP backend functionality.

---

69. FRONTEND STRUCTURE

```
frontend/
├── app/
│   ├── page.tsx
│   ├── auth/
│   ├── dashboard/
│   ├── profile/
│   ├── organizations/
│   ├── opportunities/
│   ├── funding-opportunities/
│   ├── calls/
│   ├── projects/
│   ├── applications/
│   ├── evaluations/
│   ├── funding/
│   ├── payments/
│   ├── documents/
│   ├── notifications/
│   └── ai/
├── components/
│   ├── ui/
│   ├── layout/
│   ├── opportunities/
│   ├── projects/
│   ├── applications/
│   ├── funding/
│   ├── payments/
│   └── ai/
├── lib/
├── hooks/
├── services/
├── types/
└── messages/
    ├── en/
    ├── fr/
    ├── rn/
    └── sw/
```

---

70. BACKEND STRUCTURE

```
backend/
├── manage.py
├── config/
│   ├── settings/
│   ├── urls.py
│   ├── asgi.py
│   └── wsgi.py
├── apps/
│   ├── accounts/
│   ├── persons/
│   ├── organizations/
│   ├── opportunities/
│   ├── calls/
│   ├── projects/
│   ├── applications/
│   ├── evaluations/
│   ├── funding/
│   ├── payments/
│   ├── documents/
│   ├── notifications/
│   └── ai_assistant/
└── tests/
```

Audit functionality can be implemented as a dedicated audit module/app if appropriate.

---

71. API DESIGN

Use RESTful APIs.

Examples:

```
POST   /api/auth/register/
POST   /api/auth/login/
POST   /api/auth/logout/

GET    /api/opportunities/
GET    /api/opportunities/{id}/

GET    /api/funding-opportunities/
GET    /api/funding-opportunities/{id}/
```

```
GET    /api/calls/
GET    /api/calls/{id}/

GET    /api/projects/
POST   /api/projects/
GET    /api/projects/{id}/
PATCH  /api/projects/{id}/

GET    /api/applications/
POST   /api/applications/
GET    /api/applications/{id}/
POST   /api/applications/{id}/submit/

GET    /api/evaluations/
POST   /api/evaluations/

GET    /api/funding/
GET    /api/funding/{id}/

GET    /api/disbursements/
GET    /api/payments/
POST   /api/payments/

GET    /api/documents/
POST   /api/documents/

GET    /api/notifications/
PATCH  /api/notifications/{id}/read/

POST   /api/ai/chat/
```

Use pagination, filtering, validation, permissions and consistent error responses.

---

72. SEARCH AND FILTERING

Use Django REST Framework filtering capabilities.

Support:

- keyword search
- opportunity type
- sector
- location
- organization type

- funding range
- deadline
- status

The frontend should provide a polished search/filter UX.

---

73. DASHBOARDS

The dashboard must adapt to the authenticated user's role.

Applicant dashboard

Show:

- applications
- project progress
- upcoming deadlines
- funding status
- notifications

Organization user dashboard

Show:

- organization information
- published opportunities
- calls
- applications
- evaluations
- funding
- payments

Evaluator dashboard

Show:

- assigned applications
- pending evaluations
- completed evaluations
- evaluation criteria

Administrator dashboard

Show:

- users
- organizations
- opportunities
- applications
- funding
- payments
- documents
- audit activity

---

74. RESPONSIVE DESIGN

The entire platform must be responsive.

Support:

- mobile
- tablet
- laptop
- desktop
- large screens

The original premium layout must remain visually coherent at all breakpoints.

---

75. ACCESSIBILITY

Follow accessible design principles.

Implement:

- semantic HTML
- keyboard navigation
- visible focus states
- accessible labels
- appropriate contrast
- screen-reader-friendly controls
- accessible forms
- meaningful error messages
- responsive text

76. DESIGN SYSTEM

Create reusable components.

Examples:

```
Button
Input
Select
Textarea
Checkbox
Radio
Modal
Dialog
Dropdown
Tabs
Badge
Card
Table
Pagination
Breadcrumb
Toast
Alert
Skeleton
Tooltip
Avatar
Progress
FileUpload
DatePicker
SearchBar
FilterPanel
```

Avoid duplicating UI implementations.

---

77. STATE MANAGEMENT AND DATA FETCHING

Use:

- TanStack Query for server state
- React hooks for local state
- React Hook Form for forms
- Zod for validation

Avoid unnecessary global state.

---

78. SECURITY

Implement:

- JWT authentication
- password hashing
- RBAC
- object-level permissions where necessary
- CSRF protections appropriate to the authentication architecture
- rate limiting
- secure file handling
- input validation
- SQL injection protection through Django ORM
- secure secrets management
- audit logs
- HTTPS in production

Never expose sensitive data unnecessarily.

---

79. MVP SCOPE AND FUTURE ROADMAP

The following are **not active MVP backend modules**:

- advanced mentorship marketplace
- expert marketplace
- trainer marketplace
- advanced training
- job recruitment
- investment management
- complex loan management
- complex banking/mobile-money integrations
- advanced messaging
- publications
- testimonials management
- advanced analytics/reporting
- advanced donor management
- advanced AI automation

They may appear in the original premium frontend as:

- Coming Soon

- Future
- Preview
- Roadmap

but must not be represented as fully operational backend functionality.

---

80. DEVELOPMENT OUTPUT

When implementing this project:

1. Preserve the original premium frontend layout and behavior.
2. Keep the exact selected technology stack.
3. Implement the latest MVP backend.
4. Do not introduce unnecessary technologies.
5. Do not invent unsupported backend features.
6. Use complete production-oriented files.
7. Clearly identify where every file belongs.
8. Provide comments for important code blocks.
9. Maintain consistency between frontend and backend APIs.
10. Validate all data server-side.
11. Implement secure authentication and authorization.
12. Use PostgreSQL as the production database.
13. Use Redis and Celery for asynchronous/background operations.
14. Use object storage for uploaded documents.
15. Keep financial operations transactional and auditable.
16. Keep AI grounded in official platform data.
17. Keep the platform multilingual in English, French, Kirundi and Kiswahili.
18. Make the UI responsive and accessible.
19. Build reusable components.
20. Test each module before moving to the next phase.

The final product must feel like the original premium platform concept while implementing the **latest Global Opportunities MVP as its real backend and business foundation**.